

BITTENSOR MINING OPERATIONS

Infranex BT Operator Handbook

A field guide to the Bittensor intelligence and mining operations platform: what every view does, how the Judge-to-Apply loop reaches your GPUs, and the routines that keep miners competitive.

Infranex Operations

Platform analysis and operating procedures

SEPTEMBER 2026 · EDITION 1

Table of Contents

Chapter 1 - What Infranex BT Is	1
Chapter 2 - The Core Operating Loop	2
2.1 The Node Daemon: the hinge	2
2.2 The transport ladder, honestly reported	3
2.3 Surviving reboots and automating installs	3
Chapter 3 - Intelligence Group (Views 01-05)	3
3.1 Dashboard (01)	4
3.2 Opportunities (02)	4
3.3 Subnets (03)	4
3.4 Judge Lab (04)	4
3.5 CPU Guide (05)	5
Chapter 4 - Operations Group (Views 06-11)	5
4.1 GPU Catalog (06)	5
4.2 Deployments (07)	6
4.3 DevOps Engine (08)	6
4.4 My Miners (09)	7
4.5 Monitoring (10)	7
4.6 Optimization (11)	7
Chapter 5 - Platform and Administration (Views 12-15)	7
5.1 Analytics (12)	7
5.2 System and Errors (13)	8
5.3 Runbook (14)	8
5.4 Access Control (15)	8
Chapter 6 - Overall Assessment	8
6.1 Watch items and mitigations	9
Chapter 7 - Operating Instructions	10
7.1 One-time setup	10
7.2 Daily routine, about five minutes	10

7.3 Weekly routine, about fifteen minutes 10

7.4 When something breaks 11

7.5 Security hygiene 11

Chapter 1 • What Infranex BT Is

Infranex BT is a Bittensor intelligence and mining operations platform: a single web application that answers the three questions every mining operator faces continuously. Which subnets are worth mining right now? How do I get a miner running on real GPU hardware without hand-rolling the infrastructure? And once it is running, is it actually competitive, earning, and staying healthy? The platform wraps live network intelligence, a GPU deployment engine, an on-host control daemon, and a full operations cockpit into one authenticated dashboard, so the operator stops juggling SSH sessions, spreadsheets, and block explorers.

The intelligence layer streams from three live sources, summarized below. The Bittensor Finney chain connection tracks sixteen subnets with miner counts, TAO staked, alpha prices, tempo, and emission status on a thirty-second refresh. CoinGecko supplies the TAO/USD price, market capitalization, and 24-hour change at the same cadence. RunPod's GraphQL API contributes live GPU rental offers spanning 45 datacenter types, with both spot and on-demand pricing, refreshed every minute.

15 dashboard views	4 view groups	30s chain data refresh	60s fix-to-miner latency
------------------------------	-------------------------	----------------------------------	------------------------------------

Source	Data provided	Refresh
Bittensor Finney chain	16 subnets: miner counts, TAO staked, alpha prices, tempo, emission status	30 seconds
CoinGecko	TAO/USD price, market cap, 24h change	30 seconds
RunPod GraphQL	21 live GPU offers across 45 types, spot and on-demand	60 seconds

Live data sources and refresh cadence

Under the hood the stack is deliberately modern and boring in the right places: Next.js 16 with the App Router and standalone output, TypeScript 5, Tailwind CSS 4 with shadcn/ui components, Prisma ORM over SQLite in development and PostgreSQL in production, TanStack Query for server state, and Recharts for visualization. Authentication is a five-operator ID-plus-access-code gate with HMAC-signed session cookies, script-hashed credentials, and in-memory brute-force throttling. The design language is a dark editorial theme built on Fraunces, Inter Tight, and JetBrains Mono.

What makes Infranex BT more than a dashboard is that it can act. The deployment engine provisions GPU servers and launches miners through an eleven-state lifecycle. The Node Daemon, installed on each GPU pod, executes platform-approved commands directly on the host. Judge Lab scores a miner against a subnet's validator profile and pushes corrective configuration with one click. The rest of this handbook

walks through every view in the navigation, explains how the pieces connect, and closes with the daily and weekly routines that keep a fleet competitive.

Chapter 2 • The Core Operating Loop

Everything in the platform serves one loop. First, research: Opportunities and Subnets rank where capital and hardware earn best. Second, deploy: the GPU Catalog and deployment engine rent a pod and launch the miner. Third, connect: the Node Daemon is installed on the pod so the platform can reach the machine. Fourth, verify: Judge Lab simulates the subnet's validator expectations and scores the live miner. Fifth, operate: Monitoring and the DevOps Engine watch health, drift, and emissions around the clock. Sixth, learn: Analytics, the Runbook, and the error log turn incidents into procedure. Each stage feeds the next, and the loop repeats for as long as the hardware earns.

The daemon is the hinge. Without it the platform can only see platform-side state; with it, Judge Lab fixes and DevOps actions genuinely reach the machine, typically within sixty seconds of the click.

2.1 The Node Daemon: the hinge

The daemon is a zero-dependency Python 3.8+ script built entirely on the standard library, generated per deployment with an embedded HMAC-SHA256 secret. It registers with the platform once, reports telemetry every sixty seconds, and polls a signed command queue on the same cadence. Because both the telemetry and command channels are HMAC-signed, a remote GPU pod never needs a browser session or cookie to participate; the install route, by contrast, stays session-guarded so only a logged-in operator can mint daemon identities.

Three command types are understood on the host. A restart command safely relaunches the miner process. A status probe returns fresh telemetry on demand. The workhorse is `apply_config`: it appends environment exports to the host `env` file, optionally overrides the miner launch command, then restarts the miner under the new profile. That single primitive is what carries Judge Lab fixes and runtime optimizations from a browser click to a running GPU process.

2.2 The transport ladder, honestly reported

Every action that tries to reach a miner travels one of three rungs, and the platform reports in its result note which one actually ran. If a daemon is registered and reachable, the command is queued and genuinely executed on the host. If the deployment is a mock, the change is simulated so the operator can rehearse without touching a real GPU. If the deployment is real but has no daemon installed, the configuration is updated platform-side only, and the note says so plainly, with a prompt to install the daemon. This honesty rule is deliberate: an operator should never wonder whether a fix landed on silicon or only in a database.

Every apply is also reversible by construction. Before any change is pushed, the engine snapshots the deployment configuration as a revision, so an over-aggressive fix is one click away from rollback in the Deployments view. Revisions record the cause, including judge-fix entries, and restore the prior environment exactly.

2.3 Surviving reboots and automating installs

The daemon installer writes the script and then picks the strongest supervisor available. On virtual machines and bare metal with systemd as PID 1, it installs a service unit with `Restart=always`, a fifteen-second restart delay, and network-online ordering, then enables it for boot; the daemon therefore survives pod reboots without operator intervention. In containers such as RunPod or Vast instances where systemd is absent, it falls back to a bash watchdog loop that relaunches the daemon within fifteen seconds of any crash. The installer is idempotent: re-pasting the same setup command updates the daemon in place, kills stray processes from older installs, and prints which launcher it chose. An uninstall command removes everything cleanly.

For fleets, the same setup command is self-contained enough to automate at the provider level. Paste it once into a RunPod template start script, a Vast.ai on-start script, or Oracle and Hetzner cloud-init user data, and every pod born from that template installs the daemon on first boot and refreshes it on every restart. The one rule is one template per deployment, because the embedded daemon identity is per deployment.

Chapter 3 · Intelligence Group (Views 01-05)

The Intelligence group answers where and what to mine. Its five views move from a thirty-second situational glance down to subnet-level mechanics, ending with the Judge, which is the platform's sharpest edge. Read them in order when evaluating a new subnet, and rely on Dashboard and Opportunities day to day.

3.1 Dashboard (01)

The Dashboard is the thirty-second health glance: the live chain subnets table, the TAO price, the count of live miners, and the average score across the fleet. It exists to answer one question before the operator dives deeper: is the world normal today? If chain data or prices look frozen or absurd, the problem is upstream data, not your miners, and the System view will confirm it.

3.2 Opportunities (02)

Opportunities ranks subnets by profitability: emission share against competition against your own cost lines, with editable settings for the minimum net-profit target and the GPU rental assumptions. It is the decision filter for where the next dollar of hardware should go. The ranking is only as honest as the cost inputs, so keep the rental and overhead assumptions current; garbage assumptions in, garbage ranking out.

3.3 Subnets (03)

The Subnets view is the deep sheet: per-subnet miner counts, TAO staked, alpha prices, and emission status, searchable and filterable, with per-subnet requirement overrides that the operator can edit. Those overrides matter more than they first appear, because they feed the expectations the Judge simulates against. Treat this as the know-your-battlefield view and keep the requirement data current for any subnet you actually mine.

3.4 Judge Lab (04)

Judge Lab is where the platform stops advising and starts operating. Pick a subnet, and it builds a validator-profile simulation, scores the miner across six dimensions: response quality, response speed, resource efficiency, throughput, price, and availability, then composes a composite score with a percentile estimate against the subnet field. Each scored dimension with an automated recipe carries an Apply button that pushes the corrective environment to the live miner through the daemon, after snapshotting a rollback revision. The loop closes when the operator re-runs the Judge a few minutes later and measures the gain.

Dimension	On-host change applied	Expected effect
Response quality	INFANEX_QUANT=fp8 plus INFANEX_SELF_VERIFY=1	Higher-fidelity quant tier plus self-verification toward the quality bar
Resource efficiency	INFANEX_QUANT=awq4	About half the VRAM freed so a stronger model fits the same GPU
Throughput	INFANEX_RUNTIME=vllm with chunked prefill	Two to four times throughput under concurrent validator load
Response speed	INFANEX_RUNTIME=tensorrt-llm	Up to three times lower time to first token

Dimension	On-host change applied	Expected effect
Price	INFANEX_ASK_PRICE_USD=target	Reprice into the competitive band; wrapper reprices on restart
Availability	None (manual by design)	Uptime is infra-level; no safe env-only recipe exists

Judge Lab fix recipes: dimension, on-host change, expected effect

The boundaries are as deliberate as the features. Availability is manual-only because uptime is multi-region infrastructure, not an environment variable. The price fix derives its target from the run's own price math and is sanity-clamped server-side. And the result note always names the transport actually used, so a simulated apply is never mistaken for a real one.

3.5 CPU Guide (05)

The CPU Guide collects hand-verified repositories and mechanics for CPU-mineable subnets, with copy-ready setup steps per network. It is honest about scope: it does not provision CPU pods and it does not submit miners for you. Treat it as curated documentation for the CPU niche, not as automation, and verify repo state before committing hardware.

Chapter 4 · Operations Group (Views 06-11)

The Operations group is mission control: rent, deploy, supervise, watch, and optimize. These six views are where hardware gets committed and where most of the platform's automation acts. The DevOps Engine in particular is the mines-while-you-sleep layer, and its change feed deserves a daily read.

4.1 GPU Catalog (06)

The GPU Catalog lists live RunPod offers with spot and on-demand pricing across 45 datacenter GPU types, filterable by price, memory, and class. Each card opens the guided deploy wizard with that GPU preselected, so research turns into rental in one motion. Spot pricing is the margin maker, but spot means eviction risk, which is exactly why the daemon's watchdog supervision and the deployment engine's re-provisioning matter.

4.2 Deployments (07)

Deployments is the engine room. Each deployment runs an eleven-state lifecycle from requested to running to stopped, with a config builder covering the docker image, miner command, ports, and environment variables, plus restart, stop, and kill controls. Every configuration change lands in a revision history with one-click rollback, including snapshots taken automatically before Judge fixes. The Install daemon action opens a dialog that generates the daemon script with the platform URL and a once-embedded secret, and hands the operator a copy-paste setup command that writes the daemon and its auto-start service, plus a script-only variant and an uninstall command.

4.3 DevOps Engine (08)

The DevOps Engine is the automation board: fourteen trigger kinds watch the fleet and log every decision with reasons into a live change feed. The operator's job is not to babysit it but to read the feed daily and act on the escalations. The vocabulary below is the same one the Runbook documents in depth.

Trigger	What it watches for	Operator response
GPU HEALTH	Thermals, utilization, and failure signals on rented hosts	Check the deployment card; expect auto-escalation on repeat
SUBNET DRIFT	Subnet requirements or validator profile shifted	Re-run the Judge; consider a re-quant or reprice
DEREG-RISK	Deregistration risk rising on a subnet	Weigh switching before emissions stop
ARBITRAGE	Another subnet or GPU pays more for the same hardware	Review the Optimization suggestion
RUNTIME OPT	On-host runtime tuning available	Apply with one click; revision is auto-snapshotted
PROBE FAIL	Service probes failing	Inspect Monitoring; restart if persistent
SERVICE LATENCY	Query latency degrading	Check load; consider the speed recipe
QUERY DROUGHT	Validator query volume falling	Expect a revenue dip; reassess the subnet
ESCALATION	Repeated failures escalated to a human	Intervene now; do not wait it out
UPSTREAM	Upstream platform or dependency issue	Confirm at System and Errors
BENCH REGRESS	Benchmark regression detected	Re-benchmark; inspect recent changes
RE-SYNC	State re-synchronized with the chain	Informational
SCALE	Scaling action recommended	Review capacity against revenue
KILL	Deployment killed by policy	Investigate the cause before restarting

DevOps trigger vocabulary and expected operator responses

4.4 My Miners (09)

My Miners is the profit-and-loss view per machine: monthly cost, estimated monthly revenue, and progress toward the target for every miner in the fleet. A machine whose estimated revenue sits below its cost for days is a decision waiting to be made: re-judge it, reprice it, move it, or kill it. This view exists so those decisions come from numbers, not vibes.

4.5 Monitoring (10)

Monitoring answers the alive question with operational telemetry: started miners, daily emission, monthly cost, uptime, and latency. It is deliberately distinct from the DevOps Engine, which answers the competitive question. When something looks wrong, Monitoring tells you whether the process is up and responding before you start blaming the strategy.

4.6 Optimization (11)

Optimization is the what-should-I-do-next view: switch GPU, switch subnet, or apply a runtime tune, each with expected impact and each executed through the same snapshot-then-push pipeline as the Judge. Use it when the Judge is satisfied but profits are not, and read its suggestions against the Opportunities ranking before committing hardware moves.

Chapter 5 · Platform and Administration (Views 12-15)

The last group covers the platform itself: the aggregate numbers, the plumbing, the procedures, and the people. These views are visited less often than Operations but they are the first stop when something behaves strangely, and they are where access and accountability live.

5.1 Analytics (12)

Analytics aggregates platform-level summary numbers: spend, earnings history, and trends over time. It is the weekly review surface, the place to confirm that the aggregate direction matches the per-machine story told by My Miners. If the two disagree, trust the machine-level view and go find the deployment that is dragging the average.

5.2 System and Errors (13)

System and Errors runs live health checks for every dependency the platform has: the chain RPC, the price feed, the RunPod API, and the GPU offer pipeline, plus a client-side runtime error log with resolution steps. Make it the first stop in any incident, because an outage in a data source looks exactly like a miner problem from the outside, and ten minutes of triage here saves an hour of pointless miner restarts.

5.3 Runbook (14)

The Runbook documents every trigger kind the DevOps Engine can fire, what it means, and what to do about it. It is the manual for the automation. Keep it open in a second tab during the first weeks of operation; by the second month the vocabulary will be muscle memory and the Runbook becomes the reference you hand to the next operator.

5.4 Access Control (15)

Access Control manages the five-operator base: one administrator and four member roles spanning operators and analysts. Codes are stored twice, once as a one-way script hash used for verification and once encrypted so the admin panel can display the current codes without any chat transcription. Failed and successful sign-ins are audited, and the login route throttles repeated failures per address. Regenerate any code that has ever been pasted into a chat, and deactivate rather than delete accounts that leave the team.

Chapter 6 · Overall Assessment

Judged as an operations tool rather than a demo, the platform's defining quality is that it refuses to lie. Every push reports which transport actually carried it. Every change is preceded by a snapshot and followed by a result note that names what happened. Boundaries are printed where other products would quietly imply capability. That ethos, more than any individual feature, is what makes it safe to let the automation act on real hardware.

Strength	Why it matters
Honest transport reporting	The operator always knows whether a fix reached silicon or only the platform database
Snapshot-then-apply discipline	Every push is one rollback away; revisions record cause and restore exactly
Closed Judge-to-Apply loop	Advice turns into a signed on-host change within about sixty seconds, then re-measured
Honest boundaries	Uptime stays manual, mock applies are labeled simulated, price targets are clamped

Strength	Why it matters
Sound credential model	Script hashes plus encrypted display codes, audited sign-ins, throttled brute force
Self-healing installer	Idempotent setup survives reboots via systemd or watchdog and updates in place

Standout strengths of the platform

6.1 Watch items and mitigations

Watch item	Why it matters	Mitigation
Platform URL baked into the daemon	The daemon calls the URL it was installed with; a migration breaks the callback	Re-paste the setup command with the production URL after any move
Mock deployments tick forever	Simulated fleets look alive and pollute fleet economics	Label mocks clearly and cull them before real accounting
Spot instances get evicted	The watchdog revives the daemon, not the rental	Re-provision via the deployment engine; reserve capacity for long runs
Data-source hiccups mimic miner faults	A chain RPC or RunPod API outage looks like a miner problem	Triage at System and Errors before touching miners
Development runs on SQLite	A single file is convenient but not production-grade	Move to PostgreSQL for any shared or long-lived deployment
Credential circulation	Codes and tokens pasted into chats outlive their context	Regenerate via Access Control; rotate provider keys on a schedule

Watch items, impact, and standing mitigations

Net assessment: the platform earns its own autonomy. Because every automated action is transport-honest, revision-snapshotted, and boundary-aware, the operator can hand it progressively more authority, from read-only monitoring to one-click fixes to scheduled DevOps responses, without the usual leap of faith. The watch items above are operational hygiene, not structural flaws, and every one of them has a standing mitigation already in the product.

Chapter 7 · Operating Instructions

This chapter is the working routine: what to set up once, what to check every day, what to do every week, and what to do when something breaks. Follow it as written for the first month; after that the DevOps feed and your own economics will tell you which steps to deepen.

7.1 One-time setup

1. Sign in with your operator ID and access code, then add your RunPod API key in provider settings so the GPU Catalog and deployment engine go live.
2. In Opportunities, set the profitability inputs you believe: minimum net-profit target, GPU rental cost lines, and overhead assumptions.
3. Create the deployment: pick the subnet from Subnets or Opportunities, pick the GPU in the Catalog, confirm requirements, and deploy.
4. On the deployment card, open Install daemon, copy the setup command, and paste it as root on the pod, or into your provider start script to automate future pods.
5. Wait for the daemon chip to turn online, about sixty seconds, then run your first Judge Lab evaluation against the live subnet.

7.2 Daily routine, about five minutes

Order	View	What to check
1	Dashboard	Is the world normal: chain data fresh, TAO price sane, miners live
2	DevOps Engine	Read the change feed; act on KILL, DEREG-RISK, and ESCALATION events
3	My Miners	Cost against estimated revenue per machine; flag anything underwater
4	System and Errors	Confirm all dependency health checks are green before trusting the rest

The daily five-minute sweep

7.3 Weekly routine, about fifteen minutes

Order	View	What to do
1	Judge Lab	Run a fresh evaluation on each live subnet and read the verdict

Order	View	What to do
2	Judge Lab	Apply the recommended fixes; each push is auto-snapshotted for rollback
3	Judge Lab	Re-run the evaluation a few minutes later and confirm the score moved
4	Opportunities	Confirm the subnet is still the best home for the hardware
5	Optimization	If economics say so, execute a switch or runtime tune through the same pipeline

The weekly judge-apply-remeasure cycle

7.4 When something breaks

1. Start at System and Errors: confirm the chain RPC, price feed, and RunPod API are healthy before blaming a miner.
2. Check Monitoring: is the process up, responding, and inside normal latency?
3. Open the deployment card: read the daemon chip and recent command results for failed pushes.
4. If a recent change caused it, open revisions and roll back to the last known-good snapshot.
5. Only then restart or re-provision, and record what you found in the run notes so the next operator inherits the lesson.

7.5 Security hygiene

- Regenerate any access code that has ever been pasted into a chat, straight from Access Control.
- Rotate provider API keys on a schedule and never commit them to the repository.
- Treat daemon secrets as live credentials: an installer command pasted anywhere is a credential copy.
- Prefer fine-grained, short-expiry tokens for any automation that touches the repository or the fleet.

Run this loop for a month and the platform becomes what it was designed to be: a fleet that tells you where the money is, deploys itself onto the hardware, verifies its own competitiveness, and calls for a human only when the situation genuinely needs one. The cockpit is built; the routine above is what flies it.